UpTrack

Machine-Learning-Based Infrastructure Project
Delay-Risk Prediction System

Software Requirements Specification (SRS)

Software Engineering Laboratory — Project Documentation

Document title	Software Requirements Specification — UpTrack
Version	1.0
Date	10 September 2026
Status	Draft for academic submission
Prepared by	[Student Name / Roll No.]
Course	Software Engineering Laboratory [Course Code]
Programme / Semester	[Programme, Semester]
Institution	[Department, Institution]
Guide / Faculty	[Faculty-in-charge]

Fields in square brackets are placeholders to be completed by the student before submission.

Document Control

Revision History

Version	Date	Author	Summary of changes
1.0	10 Sep 2026	[Student Name]	First complete SRS covering the UpTrack data pipeline (implemented) and the ML prediction and dashboard modules (planned).

Approval

Role	Name	Signature	Date
Author (Student)	[Name]		
Guide / Faculty	[Name]		

Compliance with the Required SRS Format

This document follows the four-part structure prescribed for the laboratory, and adds standard professional SRS material. The table below maps the required headings to their location in this document.

Required heading	Location in this document
1. Introduction — Goal & Objectives of Software	Section 1 (Goal and Objectives: Section 1.2)
2. Information Description — Problem Description	Section 2.1
2. Information Description — Information Flow	Section 2.2
2. Information Description — Hardware Interface	Section 2.5
3. Functional Description — Process Narrative (each function)	Section 3.2 (F1–F7)
4. Design Constraints — Performance Characteristics	Section 4.1
4. Design Constraints — Behavioral Description	Section 4.2
4. Design Constraints — Validation Criteria	Section 4.3

Table of Contents

Document Control 2

Revision History . . . 2

Approval . . . 2

Compliance with the Required SRS Format . . . 2

1. Introduction 5

1.1 Purpose of this Document . . . 5

1.2 Goal and Objectives of the Software . . . 5

1.2.1 Goal . . . 5

1.2.2 Objectives . . . 5

1.3 Scope . . . 5

1.4 Intended Users and Stakeholders . . . 6

1.5 Definitions, Acronyms and Abbreviations . . . 6

1.6 References . . . 6

1.7 System Overview . . . 7

1.7.1 Current Implementation Status . . . 7

2. Information Description 8

2.1 Problem Description . . . 8

Illustrative delay picture in the current dataset (May report) . . . 8

2.2 Information Flow . . . 8

2.3 Data Requirements . . . 9

2.3.1 Source Data . . . 9

2.3.2 Data Dictionary — Extracted Dataset . . . 9

2.3.3 Derived Data and the Delay Label . . . 10

2.4 Inputs and Outputs . . . 10

2.5 Hardware Interface . . . 11

2.6 Software Interfaces and Dependencies . . . 11

3. Functional Description 12

3.1 Functional Requirements Summary . . . 12

3.2 Process Narrative for Each Function . . . 12

3.2.1 F1 — Data Collection / Import . . . 12

3.2.2 F2 — Data Preprocessing / Cleaning . . . 13

3.2.3 F3 — Project Matching Across Monthly Reports . . . 13

3.2.4 F4 — Feature Preparation . . . 13

3.2.5 F5 — Delay-Risk Prediction . . . 14
3.2.6 F6 — Result Display / Dashboard . . . 14
3.2.7 F7 — Validation / Testing . . . 15
3.3 ML / Prediction Requirements . . . 15
3.4 Dashboard Requirements . . . 16
4. Design Constraints **17**
4.1 Performance Characteristics . . . 17
4.2 Behavioral Description . . . 17
4.2.1 System States . . . 17
4.2.2 Behaviour on Events and Invalid Input . . . 17
4.3 Validation Criteria . . . 18
4.3.1 Implemented — Data Extraction (enforced today) . . . 18
4.3.2 Planned — Preprocessing, Model and Dashboard . . . 18
4.4 Non-Functional Requirements . . . 18
4.5 Assumptions and Dependencies . . . 19
4.6 Limitations . . . 19
4.7 Error Handling . . . 19
4.8 Security and Privacy Considerations . . . 19
5. Validation and Testing Requirements **20**
5.1 Testing Approach . . . 20
5.2 Test Cases . . . 20
5.3 Acceptance Criteria . . . 20
Requirement Coverage Checklist (teacher's four required sections) . . . 21

1. Introduction

1.1 Purpose of this Document

This Software Requirements Specification (SRS) describes the requirements of **UpTrack**, a machine-learning-based system that predicts the delay risk of large public infrastructure projects. UpTrack uses the monthly project-monitoring data published by the Ministry of Statistics and Programme Implementation (MoSPI), Government of India (referred to in the project brief as the PAIMANA / MoSPI data), and presents its predictions through a simple web dashboard.

The document states what the system must do, the information it handles, the functions it performs, and the constraints under which it operates. It is written for an academic Software Engineering laboratory submission and is intended to be detailed enough to guide implementation and to support a viva discussion. Where a part of the system is already built, this is stated explicitly; where a part is still planned, it is labelled *Planned* so that the current state of the project is never overstated.

1.2 Goal and Objectives of the Software

1.2.1 Goal

Goal. To build a simple, understandable system that reads the Government of India's monthly infrastructure project-monitoring reports and predicts, for each ongoing project, whether it is *likely to face a delay*, then presents that delay-risk result to a user through an easy-to-read web dashboard.

1.2.2 Objectives

The goal is broken down into the following concrete objectives:

- **O1 — Ingest official data.** Read the MoSPI monthly report PDFs (Part-II, "List of Tables") and reliably extract the table of ongoing projects into a clean, structured dataset.
- **O2 — Clean and standardise.** Convert the raw extracted text into typed fields (numbers, month/year dates) and handle missing or malformed values without inventing data.
- **O3 — Track projects over time.** Match the same project across successive monthly reports so that its history (schedule and cost changes) can be observed.
- **O4 — Prepare features.** Derive a small, interpretable set of features (for example cost escalation, physical/financial progress and schedule slippage) from the project data.
- **O5 — Predict delay risk.** Train a simple classification model that labels each project as *Likely Delayed* or *Likely On-Time*, together with a confidence score.
- **O6 — Present results.** Show the projects and their predicted delay risk on a simple web dashboard with search and filtering.
- **O7 — Validate.** Verify each stage of the pipeline and evaluate the model with standard metrics, so the results can be trusted and explained.

The current release fully meets Objectives O1 and O2 for the May report and lays the groundwork for the rest; O3–O6 are planned and O7 is partly met (the data pipeline is already self-validating). Section 1.7.1 gives the precise status of every component.

1.3 Scope

In scope. UpTrack is an offline, single-user, educational analytics tool. Its scope covers:

- extraction of the "Details of Ongoing Projects" table (Annexure XVIII) from the MoSPI monthly Part-II report PDFs;

- cleaning and structuring of that data into a project dataset;
- matching projects across monthly reports and preparing features (planned);
- training and applying a simple delay-risk classifier (planned);
- a lightweight web dashboard that displays each project and its predicted delay risk (planned).

Out of scope. UpTrack does *not* attempt any of the following: real-time or streaming data feeds; automatic download of reports from government servers; multi-user accounts, authentication or role management; financial auditing or official decision-making; prediction of the exact number of months of delay or the exact cost overrun (the system predicts a delay-risk category, not a precise figure); and coverage of projects that are not listed in the MoSPI report used as input. UpTrack is a student project and is not intended for production or policy use.

1.4 Intended Users and Stakeholders

Stakeholder	Interest in the system
Student / Developer	Builds, runs and demonstrates the system; primary operator of the data pipeline and model.
Faculty / Evaluator	Assesses the design, correctness and the student's understanding during the viva.
Dashboard viewer (end user)	A non-technical reader who opens the dashboard to see which projects are at risk of delay.
Data source (MoSPI)	Publisher of the source reports. Not an interactive user; the system only consumes its public documents.

1.5 Definitions, Acronyms and Abbreviations

Term	Meaning
MoSPI	Ministry of Statistics and Programme Implementation, Government of India — publisher of the monthly project-monitoring reports.
PAIMANA	Name used in the project brief for the MoSPI project-monitoring data source.
Annexure XVIII	"Details of Ongoing Projects" — the specific table in the Part-II report from which project records are extracted.
Project record	One row of data describing a single infrastructure project in a monthly report.
Ongoing project	A project that has been approved and is under implementation (not yet commissioned).
Delay / Time overrun	A project taking longer than its originally approved commissioning date. Measured in months.
Cost overrun	The increase of a project's anticipated cost above its originally approved cost (in Rs. crore).
Delay risk	The system's predicted likelihood that a project is or will be delayed, shown as a category and a score.
Feature	A numeric input to the model derived from a project's data (e.g. cost ratio).
CSV	Comma-Separated Values — the tabular file format used for the extracted dataset.
SRS	Software Requirements Specification (this document).

1.6 References

- MoSPI, Government of India — Monthly Flash Report on Central Sector Projects (Part-II, "List of Tables"). Source PDFs used: `April_Part-II_List_of_tables.pdf` (572 pages) and `May_Part-2.pdf` (571 pages).
- Project source code: `src/extract_may_data.py`, `src/row_splitter.py`, `src/diagnose_may_pdf.py`.
- Extracted dataset: `data/may_projects.csv` (1,817 project records, 13 columns).

- Libraries: Python 3.13, pandas 3.0, pdfplumber 0.11. scikit-learn (for the planned model) and a web framework (for the planned dashboard) are not yet installed.

1.7 System Overview

UpTrack is organised as a linear pipeline. Official monthly report PDFs are read and the ongoing-projects table is extracted into a clean dataset. Records for the same project are matched across months, a small set of features is prepared, and a simple classifier predicts each project's delay risk. The results are written to a file and shown on a web dashboard. The pipeline is deliberately simple so that every step can be understood, checked and explained.

The system is built in Python. The implemented data-extraction stage uses **pdfplumber** to read the PDF tables and **pandas** to assemble and validate the dataset. The planned model stage will use **scikit-learn**, and the planned dashboard will use a lightweight Python web framework (for example Streamlit or Flask — to be finalised).

1.7.1 Current Implementation Status

The following table is the single source of truth for what is built today versus what is planned. It is referenced throughout the document.

Component	Artefact	Status
PDF diagnostics / inspection	src/diagnose_may_pdf.py	**Implemented**
Data extraction (May report)	src/extract_may_data.py	**Implemented**
Row-repair for mangled PDF rows	src/row_splitter.py	**Implemented**
Extracted project dataset	data/may_projects.csv (1,817 x 13)	**Implemented**
Pipeline self-validation report	in extract_may_data.py	**Implemented**
Data extraction (April report)	planned April extractor	**Planned**
Full preprocessing / type casting	cleaning module	**Partial**
Project matching across months	matching module	**Planned**
Feature preparation and labelling	feature module	**Planned**
ML model (training + inference)	models/	**Planned**
Web dashboard	app/	**Planned**
Model evaluation / metrics	evaluation module	**Planned**

Implemented *= working code exists and runs.* ***Partial*** *= basic version exists inside another module; a dedicated/complete version is planned.* ***Planned*** *= designed in this SRS but not yet coded.*

2. Information Description

2.1 Problem Description

Large public infrastructure projects — highways, railway lines, bridges, power and water projects — are frequently reported as running late and over budget. MoSPI publishes a monthly report that lists every monitored central-sector project together with its approval date, original and revised completion dates, costs and expenditure. This information is published as long PDF documents (over 500 pages each) that are hard to read manually and difficult to analyse across months.

The core problem UpTrack addresses is: **given the information recorded about a project in the monthly monitoring reports, can we automatically flag whether that project is likely to be delayed, and present this clearly?** Answering this by hand for hundreds of projects is slow and error-prone. UpTrack automates the reading of the reports, learns the patterns that accompany delayed projects, and surfaces a simple risk indicator per project.

The learning problem is framed as a **binary classification**: each project is predicted to be *Likely Delayed* or *Likely On-Time*. In the extracted May dataset the outcome is naturally derivable and reasonably balanced — of the 1,790 projects that record a time-overrun value, **827 show a delay** (time overrun greater than zero months), **912 are on schedule** (zero months), and 51 report a negative value (ahead of the original date). This balance makes the problem well suited to a simple, understandable model rather than a complex one.

Illustrative delay picture in the current dataset (May report)

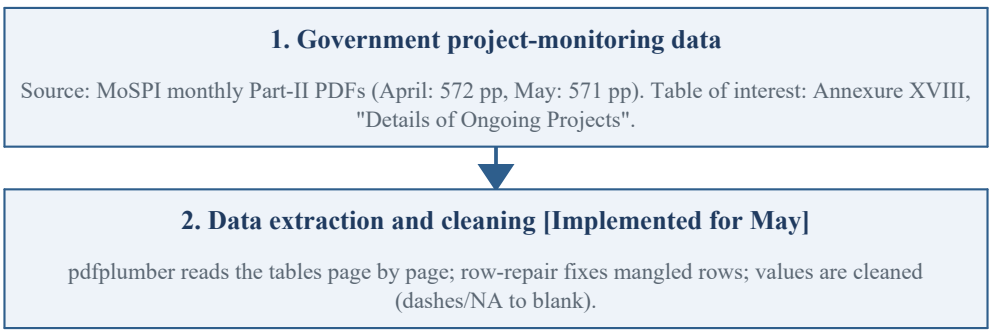
Time-overrun outcome	Projects	Interpretation
Greater than 0 months	827	Delayed (positive class)
Exactly 0 months	912	On schedule
Less than 0 months	51	Reported ahead of original date
Not recorded	27	Missing value in source
Total records	1,817	All extracted ongoing projects

2.2 Information Flow

Information moves through UpTrack in one direction, from the official documents to the dashboard. The required end-to-end flow is:

government project-monitoring data → data extraction / cleaning → project data → feature preparation → ML prediction → risk result → dashboard

The figure below expands each stage, its main input and its main output. Stages are colour-consistent with the implementation status in Section 1.7.1.



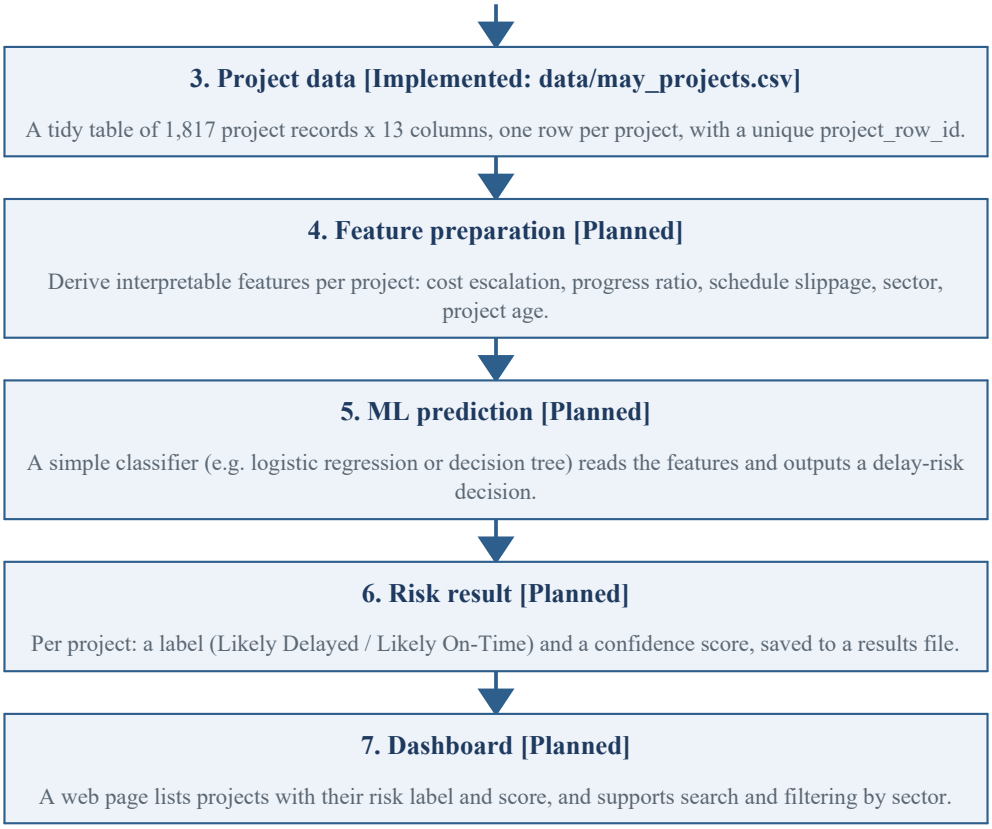


Figure 1 — UpTrack information flow (top to bottom). Bracketed labels show implementation status.

2.3 Data Requirements

2.3.1 Source Data

The only external input is the set of MoSPI monthly Part-II report PDFs, stored under `data/raw/`. Each report contains Annexure XVIII, "Details of Ongoing Projects", split into sections by sector and ministry (for example Railways, Road Transport and Highways, Power, Petroleum). Serial numbers restart within each sector, so the serial number is *not* globally unique; the extractor therefore assigns its own unique key. In the May report the ongoing-projects table spans roughly PDF pages 462–575.

2.3.2 Data Dictionary — Extracted Dataset

The extractor produces `data/may_projects.csv` with the 13 columns below. Costs are in Rs. crore; dates are month/year. Sparse columns are expected (the source often leaves "revised" values blank when a project has not been revised).

#	Column	Type	Description	Missing
1	project_row_id	text (key)	Globally unique ID assigned by the extractor (P000001 …).	0%
2	sl_no	integer	Serial number as printed (restarts per sector; not unique).	0%
3	project_name	text	Name of the project as printed in the report.	0.2%
4	date_of_approval	month/year	Date the project was approved.	0.3%
5	original_date_of_commissioning	month/year	Originally planned completion date.	0.2%
6	revised_date_of_commissioning	month/year	Officially revised completion date (if any).	66%
7	anticipated_date_of_commissioning	month/year	Currently anticipated completion date.	0.2%
8	original_cost_cr	number	Originally approved cost (Rs. crore).	0.2%

#	Column	Type	Description	Missing
9	revised_cost_cr	number	Officially revised cost, if any (Rs. crore).	76%
10	anticipated_cost_cr	number	Currently anticipated cost (Rs. crore).	0.2%
11	cumulative_expenditure_cr	number	Money spent so far (Rs. crore).	0.4%
12	cost_overrun_cr	number	Cost overrun as printed (Rs. crore).	0.5%
13	time_overrun_months	integer	Time overrun in months (basis of the delay label).	1.5%

2.3.3 Derived Data and the Delay Label

Two kinds of data are produced inside the pipeline and are *not* present in the raw source:

- **Delay label (target).** A project is labelled *Delayed* when `time_overrun_months > 0`, and *On-Time* otherwise. This label is what the model learns to predict.
- **Features (inputs).** Numeric quantities derived from the columns above, for example the cost-escalation ratio (anticipated cost / original cost), the progress ratio (cumulative expenditure / anticipated cost), the schedule gap between anticipated and original commissioning dates, the project's age since approval, and the sector.

Important design note (avoiding target leakage). The columns `time_overrun_months` and `cost_overrun_cr` in the *same* report effectively contain the answer. To keep the prediction meaningful, the planned design predicts a project's delay status using features drawn from an *earlier* monthly report (April) and never feeds the overrun columns of the target month into the model as inputs. This is why project matching across months (Section 3.2.3) is part of the design. A simpler single-month baseline may be used first for learning, with this leakage limitation clearly acknowledged.

2.4 Inputs and Outputs

	Item	Form	Produced / consumed by
Input	MoSPI monthly report PDF	PDF file in data/raw/	F1 Data collection
Intermediate	Extracted project dataset	CSV (data/may_projects.csv)	F1 → F2
Intermediate	Cleaned, typed dataset	CSV / in-memory table	F2 Preprocessing
Intermediate	Matched multi-month dataset	CSV / in-memory table	F3 Matching (planned)
Intermediate	Feature matrix + labels	CSV / in-memory table	F4 Feature prep (planned)
Intermediate	Trained model	model file in models/	F5 Training (planned)
Output	Per-project delay-risk result	CSV + on-screen	F5 → F6
Output	Dashboard view	Web page in a browser	F6 Dashboard (planned)
Output	Validation / evaluation report	console text / file	F7 Validation

2.5 Hardware Interface

UpTrack does not require any dedicated, special or custom hardware. It is a pure software application that runs on an ordinary personal computer. There are no sensors, microcontrollers, embedded devices, GPUs, scanners or other physical instruments in the system, and it does not interface with any hardware beyond the standard components of a normal computer.

The only hardware involved is the user's own computer and its standard peripherals, used in the usual way:

- **Processing and storage:** a standard CPU runs the Python pipeline; the hard disk / SSD stores the input PDFs, the dataset, the trained model and the results.
- **Input devices:** keyboard and mouse, used to run the pipeline and to operate the dashboard in a browser.
- **Display:** a normal monitor / screen shows the dashboard and console output.
- **Network:** for the planned dashboard, the browser talks to a local web server over standard networking (typically `localhost` on the same machine). No internet connection is required to run the system once the report PDFs are present.

A typical student laptop is more than sufficient. Indicative minimum specification: a modern dual-core CPU, about 4 GB of RAM (8 GB comfortable), and a few hundred megabytes of free disk space for Python, the PDFs (about 5 MB each) and the generated files. These are guidance figures for a comfortable experience, not strict hardware requirements.

Hardware element	Role in UpTrack	Special requirement?
CPU	Runs extraction, model training and inference	No — any modern CPU
RAM	Holds the ~1,800-row dataset in memory (pandas)	No — ~4 GB is ample
Disk / SSD	Stores PDFs, CSV dataset, model, results	No — a few hundred MB free
Monitor, keyboard, mouse	Operate the tool and view the dashboard	No — standard peripherals
Network interface	Browser ↔ local web server (planned dashboard)	No — localhost is sufficient
GPU / sensors / embedded HW	Not used	Not applicable

2.6 Software Interfaces and Dependencies

UpTrack interacts only with software components on the same machine:

- **Operating system / file system:** reads the PDF files and writes CSV, model and result files. Developed on Windows 11 with Python 3.13.
- **Python libraries (implemented stage):** pdfplumber (PDF table reading) and pandas (data assembly, cleaning and validation).
- **Python libraries (planned stages):** scikit-learn (model), and a web framework such as Streamlit or Flask (dashboard). These are not yet installed.
- **Web browser (planned):** any modern browser renders the dashboard served locally.

3. Functional Description

3.1 Functional Requirements Summary

The system is composed of seven functions (F1–F7). Each is expanded as a process narrative in Section 3.2. The table lists the functions, their key requirements and their status.

ID	Function	Key requirement	Status
F1	Data collection / import	Read the report PDF and extract the ongoing-projects table.	**Implemented**
F2	Data preprocessing	Clean values and cast to numbers / dates; handle missing data.	**Partial**
F3	Project matching	Match the same project across monthly reports.	**Planned**
F4	Feature preparation	Derive interpretable features and the delay label.	**Planned**
F5	Delay-risk prediction	Train and apply a simple classifier per project.	**Planned**
F6	Result display / dashboard	Show projects and predicted risk in a web page.	**Planned**
F7	Validation / testing	Check each stage and evaluate the model.	**Partial**

3.2 Process Narrative for Each Function

3.2.1 F1 — Data Collection / Import

Status: Implemented

Purpose. Turn an official monthly report PDF into a clean, structured table of ongoing projects.

Inputs. A MoSPI Part-II PDF in `data/raw/` (currently the May report).

Processing steps.

1. Open the PDF and select the page range that contains Annexure XVIII ("Details of Ongoing Projects"), excluding the tail of the previous annexure so serial numbers do not collide.
2. For each page, extract table rows with pdfplumber using a line-based table strategy, falling back to a text strategy and then to a word-position strategy if needed.
3. Repair the small number of rows that the PDF extractor mangles, using three known patterns: a serial number glued to the project name; two projects merged into one row; and an orphan data row whose serial and name are missing from the PDF text layer.
4. Assemble physical rows into one logical record per project, tracking the current sector section and skipping headers, page numbers and subtotal / grand-total rows.
5. Assign a globally unique `project_row_id` and write the 13-column dataset to `data/may_projects.csv`.

Outputs. A CSV file of 1,817 project records with 13 columns, plus a console validation report.

Error handling. If the PDF is missing the program stops with a clear message. Rows that cannot be parsed are recorded in a skip log with the reason; totals and headers are deliberately skipped; genuinely absent project names are left blank rather than being invented.

Implemented in `src/extract_may_data.py` and `src/row_splitter.py`. **Planned extension:** a matching extractor for the April report, reusing the same logic.

3.2.2 F2 — Data Preprocessing / Cleaning

Status: Partial

Purpose. Convert the extracted text fields into clean, typed values ready for analysis.

Inputs. The extracted dataset from F1.

Processing steps.

1. Strip stray characters and treat dashes and "NA"/"NIL" as missing (already done during extraction).
2. Cast cost and expenditure columns to numbers (removing thousands separators) and time overrun to an integer.
3. Parse the month/year date columns into a consistent date representation.
4. Record, but do not fabricate, missing values; keep an explicit missing-value profile per column.
5. Optionally drop or flag records that are unusable for modelling (for example, missing the fields needed to compute a feature).

Outputs. A cleaned, typed dataset (in memory and/or a cleaned CSV) with a documented missing-value profile.

Error handling. Values that cannot be parsed are set to missing and counted, never guessed. The step reports how many values in each column were converted, blank or dropped.

Partial: basic cleaning (dash / NA handling, thousands-separator removal at read time) already happens inside F1; a dedicated, complete preprocessing step with full date parsing and type casting is *planned* as a separate, testable module.

3.2.3 F3 — Project Matching Across Monthly Reports

Status: Planned

Purpose. Recognise the same project in two different monthly reports so that its history can be tracked and, crucially, so that earlier-month features can be paired with a later-month outcome.

Inputs. Two cleaned datasets from two months (e.g. April and May).

Processing steps.

1. Normalise project names (upper-case, collapse spaces, remove punctuation) to compare them fairly.
2. Match records primarily on the normalised project name within the same sector section.
3. Use supporting fields (approval date, original cost) to confirm a match and to break ties.
4. Classify each project as matched (present in both months), new, or dropped, and record the counts.
5. Produce a combined table that links each project's April record to its May record.

Outputs. A matched, multi-month dataset that pairs each project across the two reports, with match statistics.

Error handling. Ambiguous or low-confidence matches are left unmatched and reported rather than forced. Because names in the source can vary slightly between months, the matching rate is reported as a quality measure and manual spot-checks are expected.

Planned. This function enables the leakage-free prediction design described in Section 2.3.3.

3.2.4 F4 — Feature Preparation

Status: Planned

Purpose. Turn each project's raw fields into a small, interpretable set of numeric features and attach the delay label.

Inputs. The cleaned (and, for the leakage-free design, matched) dataset.

Processing steps.

1. Compute the delay label from the target month: Delayed if time overrun > 0, else On-Time.
2. Derive features from the earlier-month fields: cost-escalation ratio (anticipated / original cost), progress ratio (expenditure / anticipated cost), schedule gap in months (anticipated minus original commissioning date), project age since approval, and the sector.
3. Encode the sector as a categorical feature (e.g. one-hot) and scale numeric features if the chosen model needs it.
4. Handle any remaining missing feature values with a simple, documented rule (e.g. median or a "missing" flag).
5. Split the data into training and test sets (e.g. 80/20).

Outputs. A feature matrix with one row per project, a matching label column, and a train/test split.

Error handling. Features that cannot be computed (missing inputs) are imputed by a stated rule or the record is excluded; the count of excluded records is reported. The step asserts that no target-derived column is used as an input feature.

Planned. The feature set is intentionally small and explainable, suitable for a student project and easy to defend in a viva.

3.2.5 F5 — Delay-Risk Prediction

Status: Planned

Purpose. Learn the pattern of delayed versus on-time projects and predict a delay-risk category for each project.

Inputs. The feature matrix and labels from F4.

Processing steps.

1. Train a simple, interpretable classifier on the training set — for example logistic regression or a decision tree / small random forest from scikit-learn.
2. Apply the trained model to the test set and to any project needing a prediction.
3. For each project output a category (Likely Delayed / Likely On-Time) and a confidence score (probability) between 0 and 1.
4. Save the trained model to `models/` and the predictions to a results file.

Outputs. A trained model file and a per-project results table (id, name, predicted label, score).

Error handling. If the model cannot produce a probability for a record, it falls back to the majority-class prediction and flags the record. The methodology is deliberately kept simple; no deep-learning or ensemble complexity is required.

Planned. No accuracy figure is claimed in this document because the model has not yet been trained; target metrics are given as goals in Section 4.3 and marked TBD until measured.

3.2.6 F6 — Result Display / Dashboard

Status: Planned

Purpose. Present the projects and their predicted delay risk to a user through a simple web page.

Inputs. The per-project results file from F5.

Processing steps.

1. Load the results into a lightweight web application.
2. Show a table of projects with name, sector, key figures and the predicted risk label and score.
3. Colour or badge each row by risk (e.g. red for Likely Delayed, green for Likely On-Time).
4. Provide search by project name and filtering by sector and by risk category.
5. Show a small summary (counts of at-risk vs on-time projects, and simple charts).

Outputs. An interactive dashboard page viewable in a browser.

Error handling. If the results file is missing or empty, the dashboard shows a clear, friendly message instead of crashing. Invalid filter input returns an empty result set with an explanatory note.

Planned. The dashboard is intentionally simple (a single main page). The web framework (Streamlit or Flask) will be finalised during implementation.

3.2.7 F7 — Validation / Testing

Status: Partial

Purpose. Confirm that each stage behaves correctly and that the model's quality is measured and reported.

Inputs. Outputs of every stage.

Processing steps.

1. Validate extraction: correct column count (13), unique ids, no total/subtotal rows leaked, expected missing-value profile, and structured (not fallback) extraction on data pages.
2. Validate preprocessing: numeric fields are numeric, dates parse, and counts of converted / missing values are reported.
3. Validate matching: report matched / new / dropped counts and spot-check a sample.
4. Evaluate the model: report accuracy, precision, recall, F1-score and a confusion matrix on the test set.
5. Validate the dashboard: every project shows a label and score, filters work, and empty/error states are handled.

Outputs. A validation report (console and/or file) and, for the model, an evaluation summary.

Error handling. Any failed check is reported clearly; the extraction stage already exits with a non-zero status and a warning when a check fails, so problems are visible rather than silent.

Partial: the extraction stage already produces a detailed self-validation report (Section 4.3); model evaluation and dashboard checks are *planned*.

3.3 ML / Prediction Requirements

These requirements keep the machine-learning part simple, honest and explainable.

- **ML-R1.** The task is binary classification: predict Delayed vs On-Time for each project.
- **ML-R2.** The delay label is defined once and clearly: Delayed when time overrun > 0 months.
- **ML-R3.** Only a small, interpretable feature set is used (Section 3.2.4); no opaque or hand-tuned complex features.

- **ML-R4.** The model is a simple, well-understood classifier (logistic regression or a decision-tree / small random forest). Deep learning is explicitly out of scope.
- **ML-R5.** Target leakage is avoided: overrun columns of the predicted month are never used as inputs; the intended design predicts from an earlier month's features.
- **ML-R6.** The model is evaluated with a train/test split and standard metrics (accuracy, precision, recall, F1, confusion matrix).
- **ML-R7.** Every prediction includes a confidence score in the range 0 to 1.
- **ML-R8.** No accuracy figure is asserted until it has actually been measured; targets are goals only (see Section 4.3).

3.4 Dashboard Requirements

- **DB-R1.** Display all predicted projects in a searchable, sortable table.
- **DB-R2.** Show, per project: name, sector, original and anticipated cost, key dates, predicted risk label and confidence score.
- **DB-R3.** Visually distinguish at-risk projects (e.g. colour badge).
- **DB-R4.** Allow filtering by sector and by risk category, and search by project name.
- **DB-R5.** Show a summary panel with counts and a simple chart of at-risk vs on-time projects.
- **DB-R6.** Run locally in a browser without requiring an internet connection.
- **DB-R7.** Handle missing/empty results and bad input gracefully with clear messages.

4. Design Constraints

4.1 Performance Characteristics

UpTrack is an offline, batch-oriented tool for a single user; it does not need real-time or high-throughput performance. The figures below are **design targets** for a typical student laptop, not measured benchmarks (except where noted), and are considered comfortably achievable given the small data size (about 1,800 projects per report).

Aspect	Target	Notes
Full extraction of one report	Completes in a few minutes	One-off batch step; ~114 data pages (May).
Dataset size in memory	Trivial (~1,800 rows x 13 cols)	Well within pandas capacity on 4 GB RAM.
Model training	Seconds to under a minute	Simple classifier on a small dataset (planned).
Single prediction / inference	Under 1 second	Interactive feel on the dashboard (planned).
Dashboard page load	Under 2–3 seconds	Local server, small result set (planned).
Concurrent users	1 (single user)	Not designed for multi-user load.

Targets marked "(planned)" apply to modules not yet built and will be confirmed during testing.

4.2 Behavioral Description

UpTrack behaves as a sequence of discrete, mostly manual steps rather than an always-on service. It is deterministic: the same inputs produce the same outputs. Once a stage completes, its output becomes the input to the next stage. The system is idle between runs and performs work only when the user starts a stage (or opens the dashboard).

4.2.1 System States

State	What happens	Moves to
Idle	Waiting; no processing.	Extracting (user runs F1)
Extracting	Reading the PDF and building the dataset (F1).	Preprocessing / Idle
Preprocessing	Cleaning and typing the data (F2).	Matching / Feature prep
Matching	Linking projects across months (F3).	Feature prep
Feature prep	Building features and labels (F4).	Training
Training	Fitting the model, then evaluating (F5, F7).	Serving / Idle
Serving	Dashboard shows results and answers user actions (F6).	Idle (on close)
Error	A stage reports a failure and stops.	Idle (after user fixes cause)

4.2.2 Behaviour on Events and Invalid Input

- **Missing input file:** the affected stage stops immediately with a clear message and a non-zero exit status; no partial output is written as if it were complete.
- **Malformed / unparseable data:** the offending value is treated as missing and counted; the stage continues and reports the count.
- **Empty results in the dashboard:** a friendly "no data" message is shown instead of an error.
- **Repeated runs:** re-running a stage regenerates its output deterministically (idempotent).
- **User filter/search with no matches:** an empty list with an explanatory note is returned.

4.3 Validation Criteria

The system is considered acceptable when the following criteria are met. Criteria for implemented stages are already enforced by the extraction stage's built-in validation report; criteria for planned stages define their acceptance in advance.

4.3.1 Implemented — Data Extraction (enforced today)

- The output CSV has exactly **13 columns**.
- Every `project_row_id` is **unique**.
- **No** total, subtotal or grand-total rows leak into the data.
- Repeated serial numbers are recognised as expected (they restart per sector) and do not cause duplicate-key errors.
- Missing-value percentages per column are within expected bounds (sparse "revised" columns are allowed; core columns are near-complete).
- Data pages are read by structured table extraction, not the word-position fallback.
- The extraction program exits with a success status only when all of the above pass; otherwise it prints a warning and exits non-zero.

4.3.2 Planned — Preprocessing, Model and Dashboard

- **Preprocessing:** all cost/time columns are numeric and all date columns parse; the converted/missing counts are reported.
- **Feature prep:** the feature matrix contains no unexpected missing values after the documented imputation, and no target-derived column is used as an input.
- **Model output:** every input project receives a label in {Likely Delayed, Likely On-Time} and a score in [0, 1].
- **Model quality (targets, TBD until measured):** the model should beat the majority-class baseline (about 54% on the current data) and report accuracy, precision, recall and F1 on the test set; a reasonable initial target is accuracy of at least 0.70, to be confirmed empirically.
- **Dashboard:** every project row shows a label and score; sector and risk filters and name search return correct results; empty and error states display a clear message.

4.4 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Usability	Running a stage is a single command; the dashboard is readable by a non-technical viewer.
NFR-2	Reliability	Deterministic outputs; failures are explicit (clear message, non-zero exit), never silent.
NFR-3	Maintainability	Small, single-purpose modules with comments; clear separation of pipeline stages.
NFR-4	Portability	Runs on a standard PC with Python 3.13; no special hardware or OS features.
NFR-5	Transparency	Simple, interpretable model and features; every prediction has a score.
NFR-6	Reproducibility	Given the same PDFs and code, the dataset and results can be regenerated.
NFR-7	Data integrity	The system never fabricates data; missing source values stay missing.
NFR-8	Performance	Meets the modest targets of Section 4.1 on a typical laptop.

4.5 Assumptions and Dependencies

- The MoSPI report PDFs keep a broadly consistent layout for Annexure XVIII across months; large format changes would need extractor adjustments.
- The report PDFs are available locally in `data/raw/` (the system does not download them).
- The delay label based on the reported time overrun is an acceptable ground truth for this educational project.
- Python 3.13 with pandas and pdfplumber is available; scikit-learn and a web framework will be installed for the planned stages.
- A single user operates the tool on one machine; no multi-user or deployment concerns apply.

4.6 Limitations

- **Single report extracted so far:** only the May report is turned into data today; April extraction and cross-month matching are planned.
- **Extraction is source-dependent:** a few rows in the PDF are mangled and repaired by known patterns; an unseen anomaly could need a new rule. Three project names are genuinely absent in the source and are left blank by design.
- **Prediction is categorical:** UpTrack predicts risk (delayed or not), not the exact months of delay or exact cost overrun.
- **Model not yet trained:** no accuracy is claimed; quality will be known only after evaluation.
- **Educational scope:** results are for learning and demonstration, not for official or financial decisions.

4.7 Error Handling

Error handling follows one principle: **fail loudly and never fabricate**. Concretely:

- Missing input files stop the affected stage with a clear message and a non-zero exit status.
- Unparseable values become missing and are counted and reported, not guessed.
- The extraction stage keeps a skip log (with reasons) and a validation report, and warns when a check fails.
- The planned model falls back to the majority class and flags any record it cannot score.
- The planned dashboard shows friendly messages for empty results or bad input instead of crashing.

4.8 Security and Privacy Considerations

UpTrack uses only **public government data about projects**; it stores **no personal data** about individuals, so data-privacy risk is minimal. The main considerations are integrity and safe local operation:

- **No personal / sensitive data:** the dataset concerns public infrastructure projects only.
- **Local-only by default:** the planned dashboard runs on `localhost`; it is not exposed to the internet, so no authentication is required for this student scope.
- **Read-only source:** the original PDFs are never modified; the pipeline only reads them and writes new derived files.
- **Attribution and honesty:** the data source (MoSPI) is credited, and predictions are clearly labelled as model estimates, not official statements.
- **If ever deployed beyond localhost** (out of current scope), basic access control and input sanitisation would be required before sharing.

5. Validation and Testing Requirements

5.1 Testing Approach

Testing follows the pipeline. Each stage is checked against the validation criteria of Section 4.3 before the next stage is trusted. The implemented extraction stage is tested by its own automated validation report; planned stages will be tested with small, targeted checks and manual spot-checks appropriate to a student project.

5.2 Test Cases

ID	What is tested	Expected result	Stage
TC-1	Extract May report	1,817 rows, 13 columns, CSV written.	F1 (impl.)
TC-2	Unique keys	All project_row_id values unique.	F1 (impl.)
TC-3	No total rows leak	No total/subtotal/grand-total rows in data.	F1 (impl.)
TC-4	Repeated serials	Duplicate sl_no across sectors accepted, not an error.	F1 (impl.)
TC-5	Missing-value profile	Core columns near-complete; revised columns sparse as expected.	F1 (impl.)
TC-6	Numeric casting	Cost/time columns become numeric; bad values counted as missing.	F2 (planned)
TC-7	Date parsing	Month/year fields parse to dates; failures counted.	F2 (planned)
TC-8	Cross-month matching	Matched/new/dropped counts reported; sample verified.	F3 (planned)
TC-9	No target leakage	Overrun columns of target month absent from features.	F4 (planned)
TC-10	Label correctness	Delayed iff time overrun > 0.	F4 (planned)
TC-11	Model output shape	Every project gets a label and a score in [0,1].	F5 (planned)
TC-12	Beats baseline	Test accuracy above majority-class (~54%).	F5/F7 (planned)
TC-13	Evaluation metrics	Accuracy, precision, recall, F1, confusion matrix reported.	F7 (planned)
TC-14	Dashboard display	Projects, labels and scores shown; filters and search work.	F6 (planned)
TC-15	Empty/error states	Friendly message on missing results or no-match filter.	F6 (planned)

5.3 Acceptance Criteria

UpTrack is accepted for this laboratory when: (a) the extraction stage passes TC-1 to TC-5 automatically (already achieved for the May report); (b) the planned stages, once implemented, pass their test cases TC-6 to TC-15; and (c) the dashboard demonstrates a per-project delay-risk result end to end. Until the model is trained and evaluated, no specific accuracy figure is asserted; the target in Section 4.3 stands as a goal to be confirmed by measurement.

Requirement Coverage Checklist (teacher's four required sections)

Required section	Provided?	Where
1. Introduction — Goal & Objectives of Software	Yes	Sections 1, 1.2
2. Information Description — Problem Description	Yes	Section 2.1
2. Information Description — Information Flow	Yes	Section 2.2, Figure 1
2. Information Description — Hardware Interface	Yes	Section 2.5
3. Functional Description — Process Narrative (each function)	Yes	Section 3.2 (F1–F7)
4. Design Constraints — Performance Characteristics	Yes	Section 4.1
4. Design Constraints — Behavioral Description	Yes	Section 4.2
4. Design Constraints — Validation Criteria	Yes	Section 4.3

— End of Software Requirements Specification —